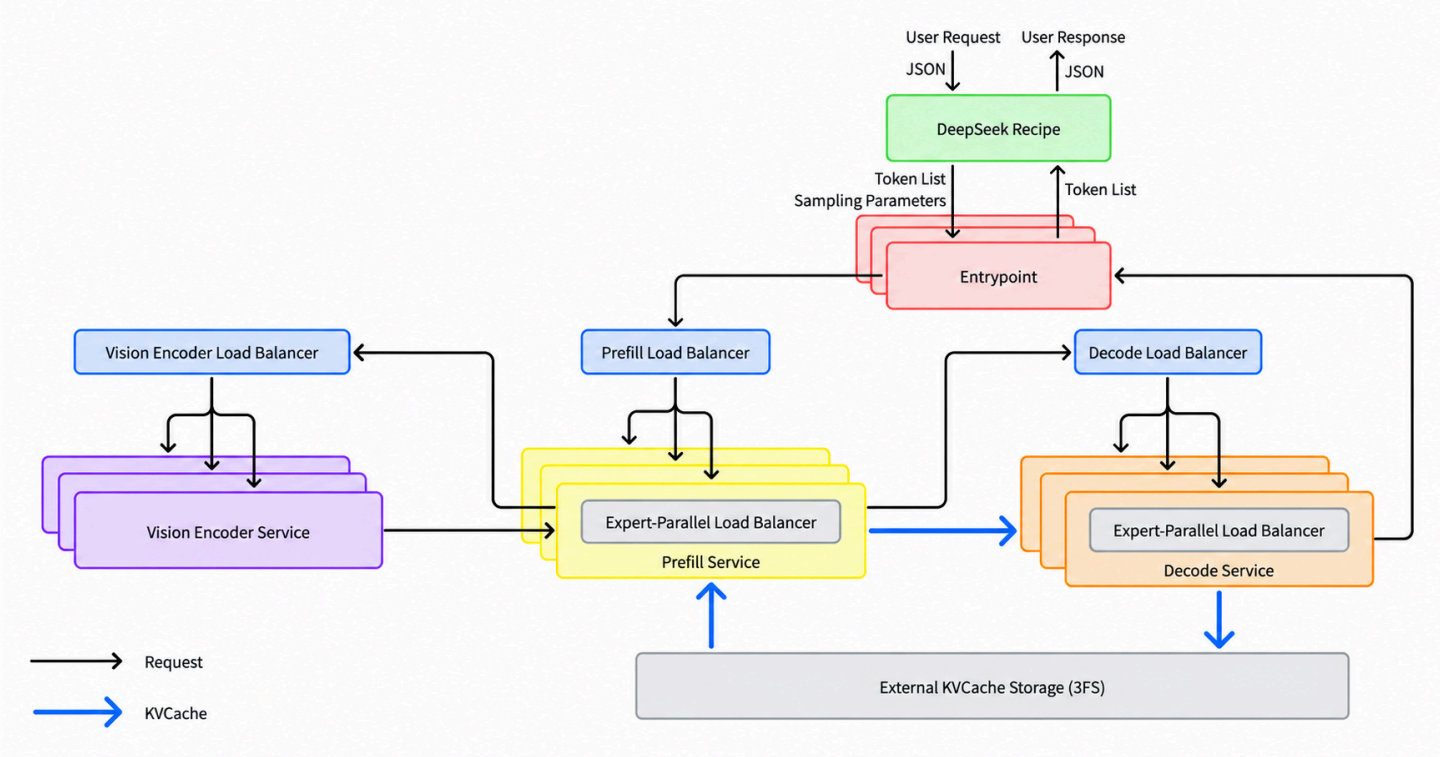
DeepSeek V4.1 推理系统拆解

DeepInfer 开源代码与部署 Q&A 解读

要点

1. Prefill 路由不做任何 cache aware，前缀命中靠池化的 KV 存储
2. 重点是几个 load balancer
3. EPLB 有重排和冗余两个手段，实测收益几乎全来自重排
4. DSpark 变长验证实测结果

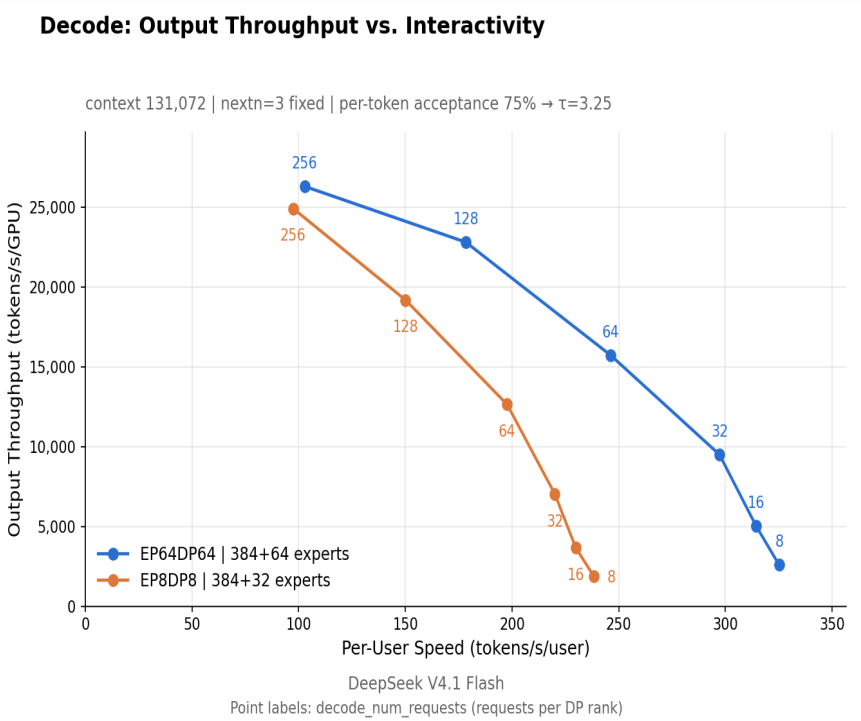
EPD 分离，P 和 D 共享一个池化的 KV 存储



EPD 即 Encoder / Prefill / Decode。系统里还有单独的专用存储节点。图来自 DeepInfer 部署文档

Prefill 用小 EP，Decode 用 Wide-EP，暂时不开 TP 和 CP

- Prefill 是 compute bound，只要能用满本卡算力，EP4 扩到 EP8 不改善总吞吐
- Decode 大概率是 memory bound，EP 用通信换带宽，每卡留出更多显存，撑更大的 batch、放更多冗余专家
- Encoder 单卡部署，便宜的卡就够
- TP 用峰值吞吐换延迟，延迟不是大问题时尽量不开



Prefill 用 PP 还是 EP：中小部署我看好 PP，大部署看好 EP

- 文档没聊 prefill PP。理论上 PP 也不改变峰值吞吐，只有一点流水线气泡
- V4.1 跨层共用 KV cache，对 PP 不太友好；模型训成每一层都不一样，天生也对 PP 不友好
- EP 要完整做完 DeepSeek 这一套，需要不少流量和 prefill worker
- 几个 prefill worker 的中小部署，PP 更合适

开源世界没有好用的池化存储，router 只能做 cache aware

- 大部分开源 KV cache 组件都是本机的，彼此不互通
- 即使"声称"互通，实际走 TCP 或低效 RDMA，也等于不互通
- 所以 router 必须做 cache aware，追着有前缀的那张卡发



数据库领域早就走过一遍：share nothing 到存算分离

- **80–00 年代** share nothing 赢：当时硬件做池化很吃亏，协议也没优化好
- **2015 年后** 硬件和系统研究成熟，PolarDB、Redshift 等走上存算分离
- **现在** 连当年坚定 share nothing 的数据库也开始支持存算分离
- share nothing 赢在网络和协议做不好池化。现在 RDMA 标配是 8 个计算 NIC + 1 个存储/管理 NIC，带宽和延迟都不再稀缺
- 稀缺的变成了计算资源（GPU 成本最高）的弹性、利用率和运维方便



存储机很豪华，SSD 寿命是主要问题

- 每台 20 块 30.73 TB SSD，单台接近 600 TB；要求读 40 GB/s，写 5 GB/s
- 按 PCIe 4.0 时代估算，Samsung PM1733a 是接近的规格（已停产）。20 块理论上读能到 150 GB/s、写 82 GB/s，实际还要扣掉 EC、多副本等开销
- KV cache 不能挑着存，所有用户的都要写，SSD 一直承接较高的写流量，要按规格推算寿命

10+2 EC 存在 3FS 上，读一次打到 11 台机器

- 每张 GPU 对应的逻辑容量是 14.4 TB，10+2 EC 膨胀 20% 后是 17.28 TB
- 读直接给 11 个节点发请求，顺带改善尾延迟。可以读 Jeff Dean 的 *The Tail at Scale*，以及 EC-Cache（OSDI '16）
- 3FS 是 chain replication，这里应该是单副本（代码支持 chain 长度为 1）。3FS 不感知 EC 本身，主要管 placement：12 个 shard 放到 12 台机器，每台承担的 shard 要均衡
- 挂掉一台后，重建流量也要均衡。这个问题叫平衡不完全区组设计（BIBD），1930 年代就有了，最初是研究怎么在多块地上测试十几种新化肥，有点像数独
- EC 相关的计算逻辑没有开源，只看到一些相关脚本

TTL：agent 3 天，chatbot 7 天

- chatbot 隔几天再续聊的比例更高，TTL 只给 3 天会掉 0.5–1% 的命中率
- Decode 不做 offload，不受 cache 容量限制
- Decode 产生的 KV cache 也会写盘，这样输出也能被命中

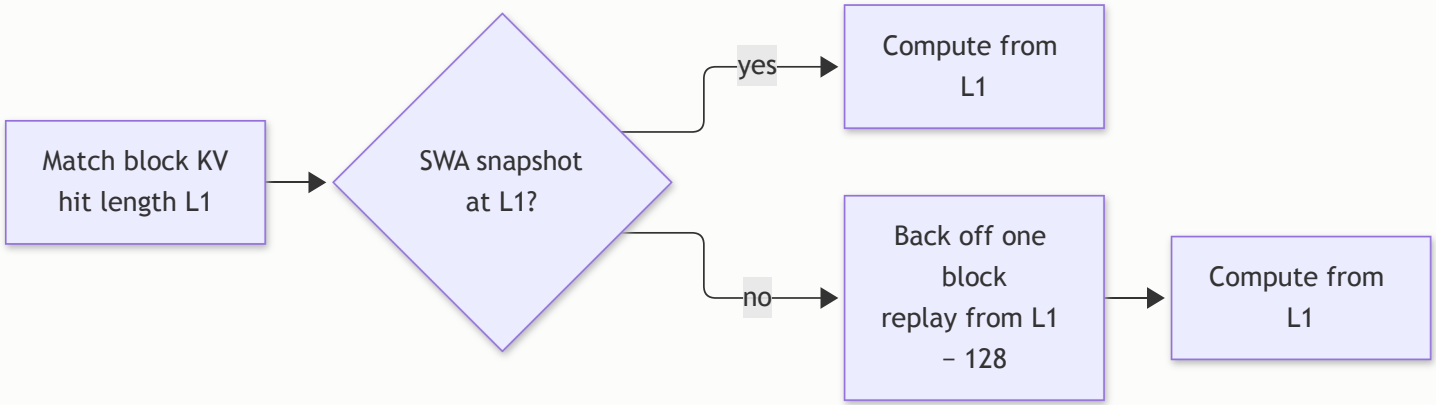
V4.1 的两种 KV 分开存

	Block KV（CSA）	SWA KV
内容	top-k + 压缩后的 KV	最近 128 个 token 的滑动窗口
显存分配	按 128 个 token（压缩前）分页，每个请求一张 block table	每个请求一个固定槽位（StateBlock）
持久化	3FS	本机内存，只在 128 的整数倍位置存快照
跨机命中	任何机器直接读	Entrypoint 记录快照在哪台机器，走 400G VLAN 拉回来

SWA 槽位启动时静态预留，不和 block KV 互相挪用

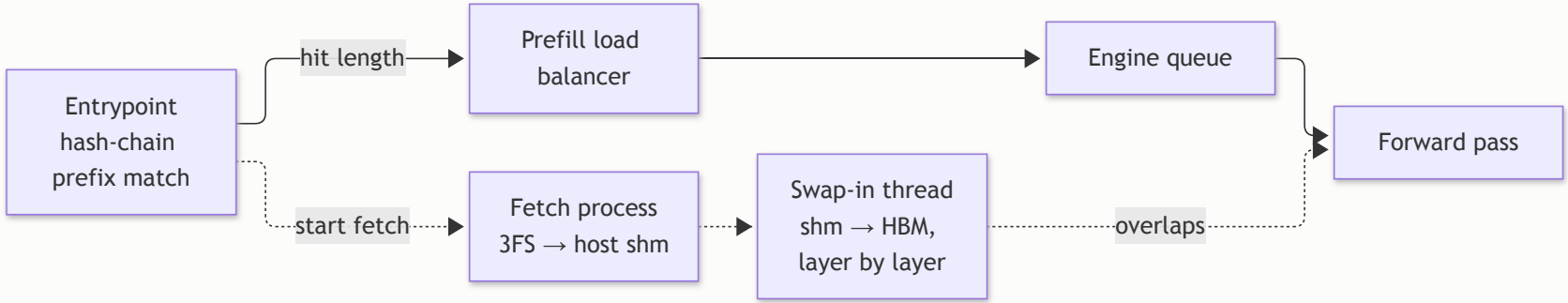
- 槽内按 `pos % 134` 当环形 buffer 用，多出来的 6 个位置给 MTP 投机失败回退
- 启动时按 `max_num_seqs × 1.2` 预留槽位，剩下的显存才给 block KV
- 调度时两边都分配得到才放行
- 每个请求的 SWA 快照约 1.4 MB（20 层 × 134 × 528 B）

命中长度看 block KV，SWA 快照缺失时重算最后 128 个 token



- 先查 block KV，得到 L1：128 的倍数，最多到 L − 134
- 重算只为重建 SWA，block KV 照样复用、不覆盖
- 重算时注意力窗口截断在这 128 个 token 内（bounded replay），得到的是近似的 SWA
- SWA 丢了的代价是常数 128 个 token，和上下文多长无关

只有匹配在关键路径上，拉取和换入与排队、前向重叠



- 匹配只查元数据，但后面要用命中长度算 compute quota，所以必须在路由前做完
- 拉取在另一个进程里异步做，和请求排队重叠
- 换入在 engine 进程内的线程里逐层做，和前向计算重叠

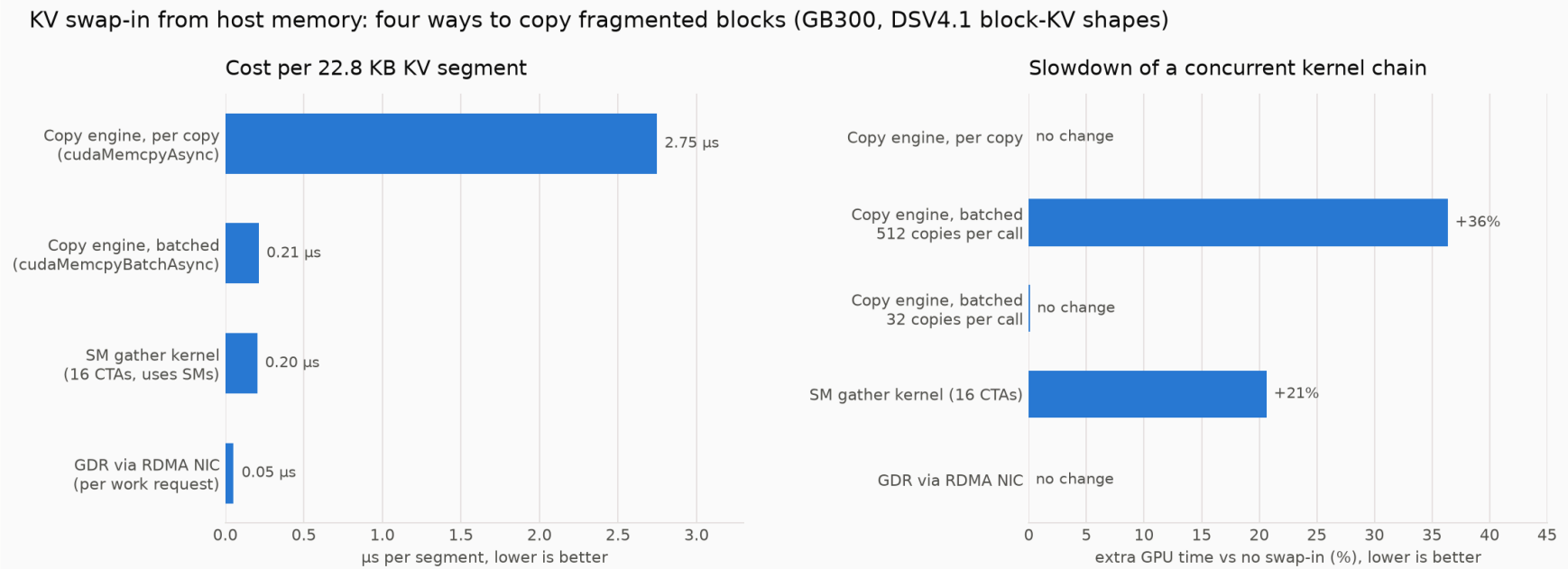
换入有四种搬法，区别在固定开销和会不会影响前向调度

Block KV 一个 block 一层是 22,784 B，命中 64K token 就是 512 个 block，源和目的地址都是散的。

1. **Copy engine 逐个拷**：每块调一次 `cudaMemcpyAsync`，每次是一个独立的 DMA 操作，不占 SM
2. **Copy engine 批量拷**：`cudaMemcpyBatchAsync`（CUDA 12.8 之后），一次调用交一批拷贝给 copy engine，不占 SM，但 batch 大小要控制，CPU 提交也有开销
3. **SM gather**：写个 kernel 用 SM 直接读 host 内存写到显存，要占 SM，而且和前向 kernel 在同一个 GPU 队列里
4. **GDR**：网卡做 RDMA write，从 shm 直接写到显存，不占 SM，换入线程也不调 CUDA API

前三种都要过 CUDA，第四种完全绕开 CUDA。DeepSeek 选的是 GDR。

小块 KV 换入用网卡（GDR）做：开销小，不占 SM

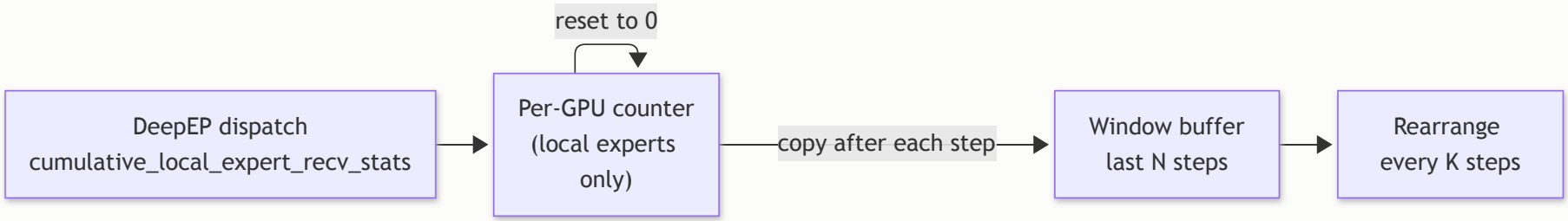


GB300 实测，DSV4.1 的 block KV 形状。左图 GDR 是网卡处理一个请求的开销，不含搬数据。

EPLB 是冗余换 LB：多占显存，换吞吐

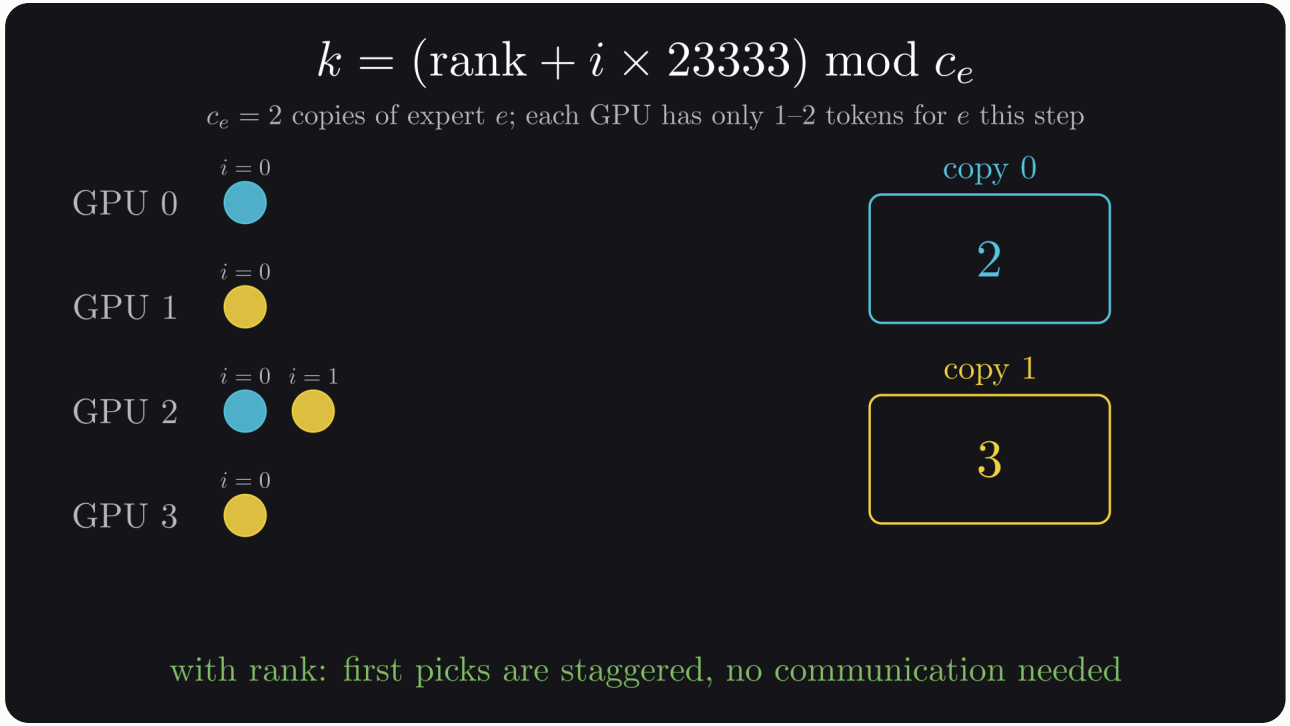
- 冗余越多收益越小，要找一个合适的点，case by case
- DeepInfer 开源的是在线 EPLB
- 两个手段：
  - **重排**：按观测到的专家负载重新分配到各卡，不增加显存。分静态（启动时指定）和动态（在线）
  - **冗余**：一个专家的权重放在多张卡上，decode 时会多读权重

在线统计：每步把各专家收到的 token 数攒进一个窗口



- DeepEP 的 dispatch 支持传一个 GPU 计数数组，参数默认是 None，传进去就行
- 每张卡只统计自己卡上那几个专家
- 每步算完拷一份到全局统计 buffer（比如窗口 128 就保留 128 步），拷完清 0

副本选择：同卡轮流分，不同卡错开起点，卡之间不通信



c_e：专家 e 的副本数；i：token 在本卡这一批里的序号；23333 是质数，用来打乱报数顺序

重排频率：两边都大约十分钟一次

- Prefill 每卡冗余 4 个专家，窗口 100 步，每 1000 步重排一次。中间 900 步的统计会被覆盖，只看最近 100 步
- Decode 每卡冗余 1 个专家，窗口 1000 步，每 10000 步重排一次
- 重排时同步搬权重：同一个 NVLink 域内走 NVLink，跨域走 NCCL `batch_isend_irecv` 点对点

EPLB 的目标：降低每一步最慢那张卡的负载

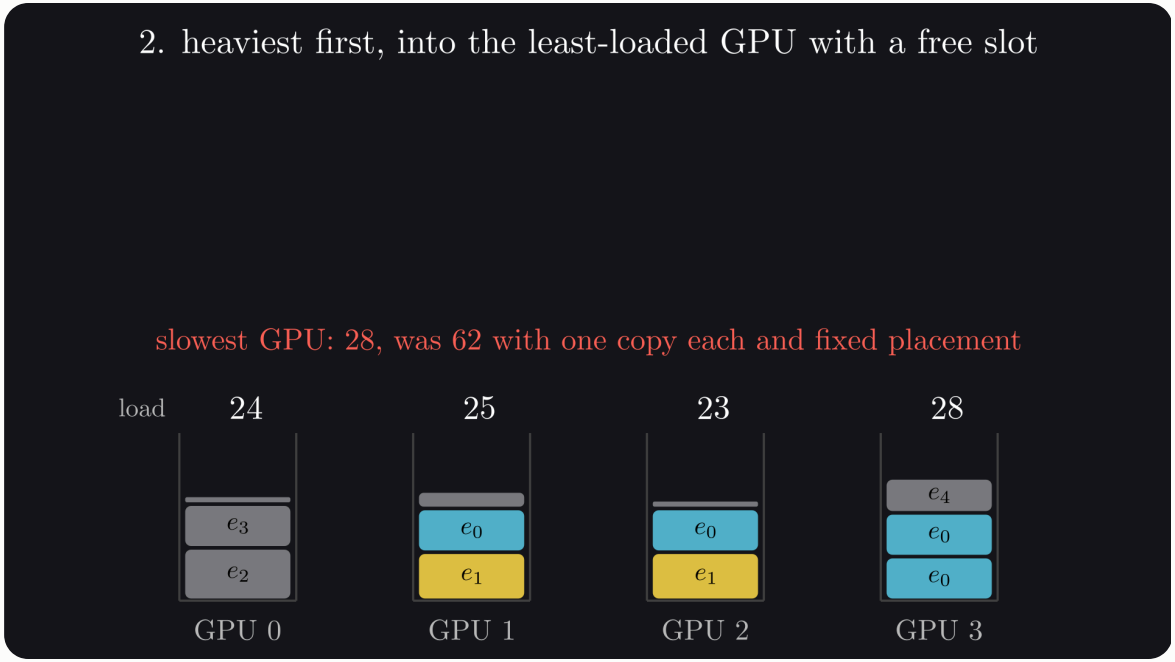
- 核心是削峰，不追求平均：每一步整个系统的速度取决于那一步最慢的卡
- 在给定 worker 和专家槽位下，让观测窗口里每一步"最慢那张卡"的总耗时最小

第一步：冗余名额一个个分给最热的专家

```
tokens_per_expert_all_diff = tokens_per_expert + arange_logical * 1e-4
for i in range(num_redundant_experts):
    score = tokens_per_expert_all_diff / logical_count     # 当前每份负载
    score_added = tokens_per_expert / (logical_count + 1)  # 多给 1 个副本后的负载
    values, indices = score.max(-1, keepdim=True)          # 每一步负载最高的专家
    values = values.expand_as(score).contiguous()
    score.scatter_(-1, indices, score_added.gather(-1, indices))
    values.scatter_(-1, indices, score.max(-1, keepdim=True).values)
    redundancy_indices = values.sum(0).argmin(-1)          # 选总分最低的专家
```

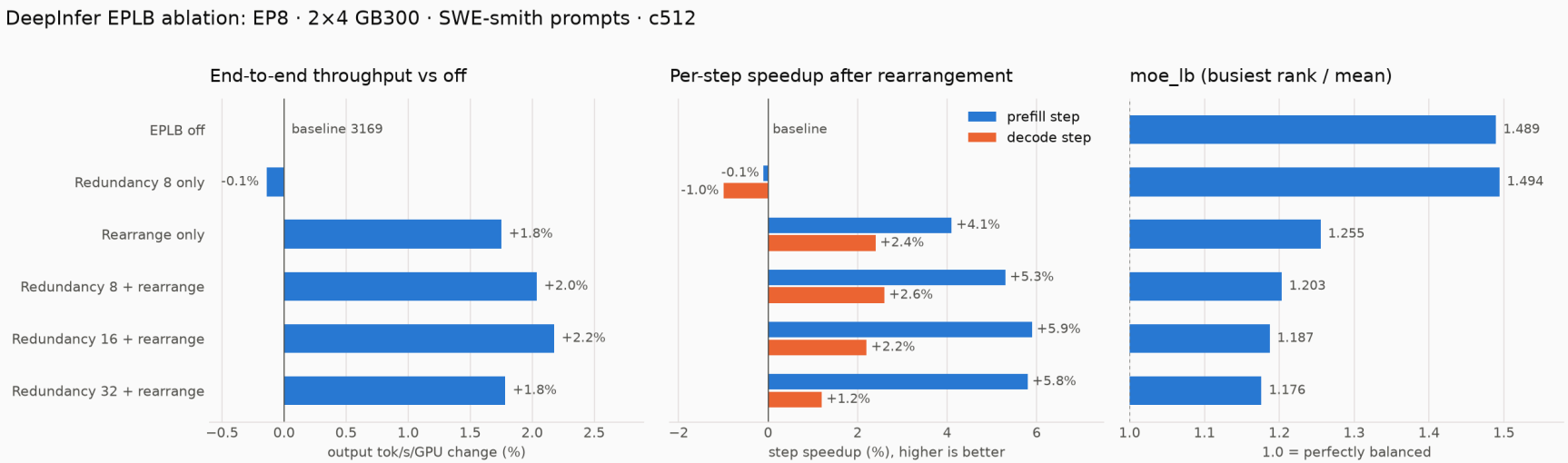
- 过热的专家放哪张卡都扛不住，只能复制。每轮给一个专家加一份副本，直到冗余名额用完，每层单独算
- 每份负载 = 专家负载 ÷ 副本数。对窗口里的每一步，找出这一步每份负载最高的专家
- 假设这份副本给专家 j：j 是某一步最热的，那一步的最高负载就降下来；不是的话，那一步不变
- 把各步的最高负载加起来，选加完后总和最小的 j
- 负载加上 1e-4 × 编号，只是为了让各专家的值互不相同，argmax 结果确定

第二步：按负载从大到小装箱（Best Fit Decreasing）



示意只画了一步的负载。实际按观测窗口里每一步都算，放到"每步最忙卡负载之和"最小的卡上。先放热专家，闲专家填缝。

实测：EPLB 端到端快约 2%，收益几乎全来自重排



Run-to-run spread <0.3%; runs with one-off JIT spikes excluded. Rearrange every 800 steps from a 200-step window; ~1 s stall each. Redundancy-only never rearranges: its step speedup covers the whole run.

2 台 GB300（EP8/DP8），SWE-smith 真实 agent prompt（p50 约 7k token），greedy 固定输出 1024 token，c512。V4.1-Flash 384 专家 top-6，每卡 48 个。只加冗余不重排反而略亏，多出来的槽位只是多读权重。EP8 规模有限，结论不一定能推到 EP64。

解决 EPLB 后，剩下的是 DP 之间的均衡（DPLB）

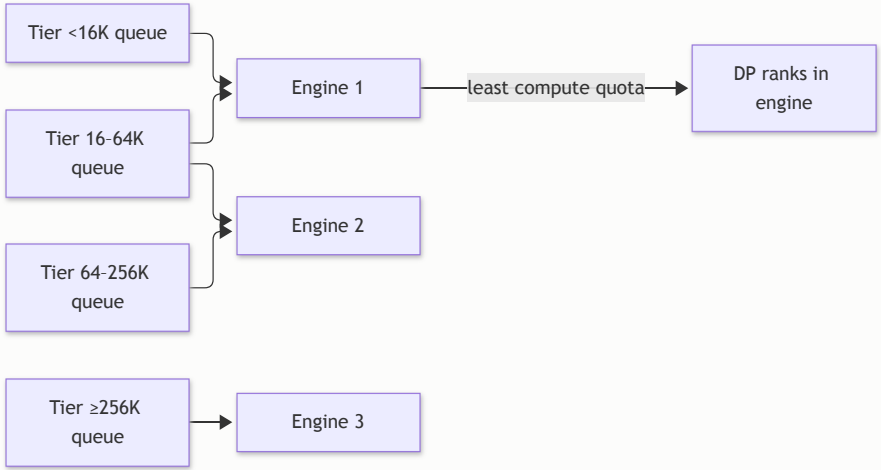


- Prefill 和 decode 都是 DP + EP，每一层都要等 attention 算得最慢的那张卡，大家再一起进 MoE 的 all-to-all
- Prefill 和 decode 的 attention 耗时由不同因素决定，表面上是两个 load balancer，实际都围绕 DPLB
- DeepSeek 自评：吞吐达到完全均衡时离线 benchmark 的 80%

Prefill：按上下文长度分 4 个桶

- <16K、16–64K、64–256K、≥256K
- 分桶的原因：attention 和上下文长度不是线性关系，同样 1K token，500K 上下文的比 10K 的慢；不同长度下 attention 和 MoE 的比例不同
- 所有机器共享 3FS，上下文增长时请求在桶之间迁移，不降低命中率
- 引擎也会按负载在桶之间迁移（都是 prefill worker），每个桶有自己的单 batch 算力配额上限
- 分桶比较生产化，开源 router 不知道有没有人做，Together 之前探索过

Prefill：引擎按算力配额主动领取请求

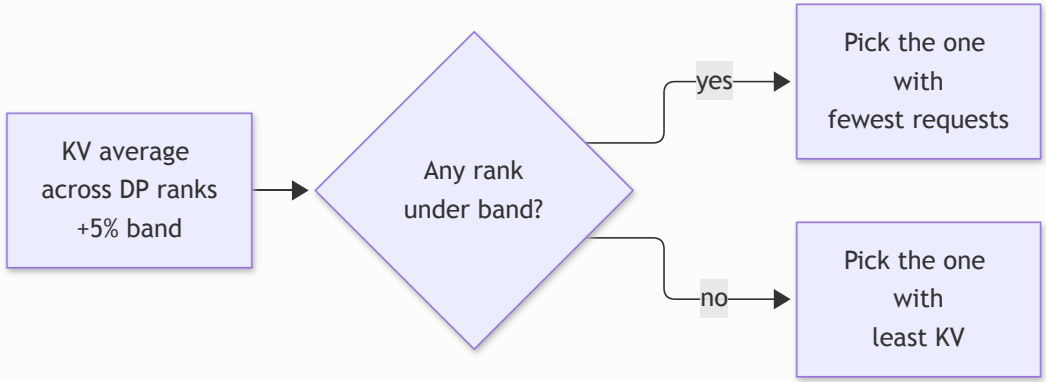


- 每个引擎有成本估算器 `cost(q, c)`，算 compute quota，约束的是 attention 耗时，不是整个 step
- 引擎按待处理请求和剩余 quota 自己拉，不做 cache aware，是 pull based。我猜是每个 tier 一个消息队列
- DP 组内按 least compute quota 派发；DP 组之间和组内都用 compute tokens 作负载指标

Prefill：按时间预算组 batch，不固定 chunk size

- DP 调度器按"时间预算"组 batch（按 DualPath 是 300 ms），再在 DP 组内 all-reduce min 收紧
- 队列是 FIFO
- 估算器的思路：离线跑出耗时表，scheduler 按表估到合适的时间去组 batch。我之前给 PP 做过一个类似的，EP 和 PP 思路一致
- DualPath 也讨论了 prefill LB，但 DeepSeek 这里没有引用，调度算法差异也挺大，留给大家想想为什么

Decode：同时平衡请求数和 KV 总长度，不分桶



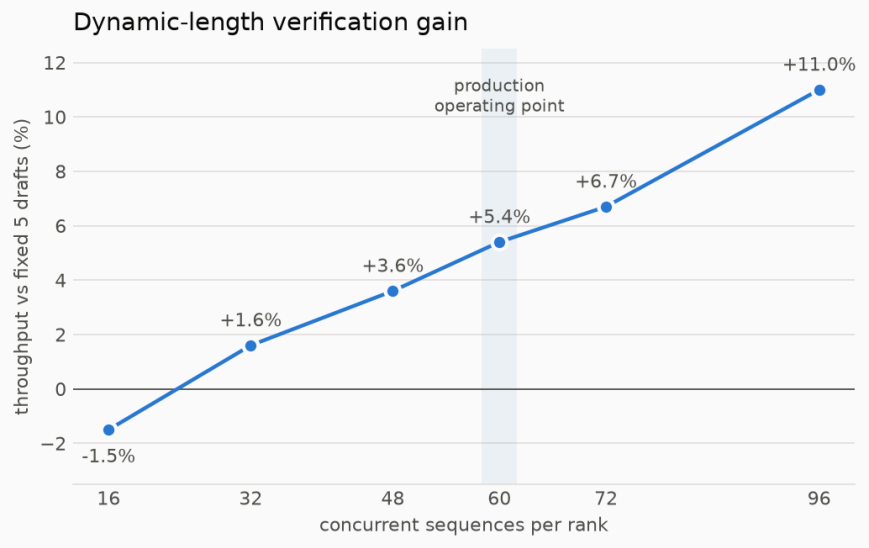
- 请求数决定 GEMM 相关算子，KV 总长度决定 attention，本质也是某种 cost
- 短请求发到并发低但 KV 总长度大的卡（填 batch 维度），长请求发到 KV 总长度小的卡
- 按 DualPath：DP 组之间用 total tokens 平衡；组内先算 KV 平均线，放宽 5%，线内选请求数最少的，都超了就选 KV 最少的

Vision Encoder 和 PD 扩缩容

- **Vision Encoder**：由 P 随机选一台机器，机器内部把图像交给活最少的 worker
- **PD 扩缩容**：prefill 看整体 compute tokens，decode 看 num running，也就是系统整体的 batch size

DSpark：按接受率决定每个请求验证几个 token

- Confidence head 预测每个 draft token 的接受率
- 大 batch 下固定投 5 个去验证代价高
- 按接受率和离线 profile 的吞吐曲线，动态决定每个请求验证几个
- 每步验证的 token 数从 5 降到 3.2–4.4，平均接受长度基本不变（3.29 vs 3.20）



数据来自 DeepInfer 部署文档，EP64。

谢谢

Q&A